

✓ Batch Normalization

Teinar modelos profundos e fazê-los convergir numa quantidade razoável de tempo pode ser uma tarefa complicada. Nesta prática, descrevemos e usaremos o [batch normalization](#) (BN), uma técnica popular e eficaz capaz de acelerar a convergência de redes profundas que, juntamente com [blocos residuais](#), nos permitiu treinar redes com mais de 100 (até mesmo 1000) camadas.

Primeiro, vamos rever alguns dos desafios práticos ao treinar redes profundas.

1. O pré-processamento de dados geralmente se prova crucial modelagem estatística. Como falado anteriormente, num geral, se padroniza a entrada para ter uma média *zero* e variância de *um*. Padronizar os dados de entrada normalmente torna mais fácil treinar modelos profundos, pois os parâmetros estão, *a priori*, em uma escala similar.
2. Para reles MLP ou CNN, enquanto treinamos o modelo, as ativações em camadas intermediárias da rede podem assumir diferentes ordens de magnitude. Os autores de [batch normalization](#) postulou que esta diferença na distribuição das ativações poderia dificultar a convergência da rede. Intuitivamente, poderíamos conjecturar que, se camada tem valores de ativação que são 100x que de outra camada, poderíamos precisar ajustar as taxas de aprendizagem de forma adaptável por camada (ou mesmo para neurônios dentro de uma mesma camada).
3. Redes mais profundas são complexas e propensas à *overfitting*. Isso significa que a regularização se torna mais importante. Empiricamente, notamos que mesmo com o *dropout*, os modelos podem cair numa situação de *overfitting*. Neste caso, devemos nos beneficiar de outras heurística de regularização.

```
import time, os, sys, numpy as np
import torch
import torchvision
import torch.nn as nn
import torch.nn.functional as F

from torch import optim
from torchsummary import summary

import time, os, sys, numpy as np

# Test if GPU is available, if not, use cpu instead
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
n = torch.cuda.device_count()
devices_ids= list(range(n))
```

```
def load_data_cifar10(batch_size, resize=None, root=os.path.join(
    '~', '.pytorch', 'datasets', 'fashion-mnist')):
    """Download the Cifar10-MNIST dataset and then load into memory."""
    root = os.path.expanduser(root)
    transformer = []
    if resize:
        transformer += [torchvision.transforms.Resize(resize)]
    transformer += [torchvision.transforms.ToTensor()]
    transformer = torchvision.transforms.Compose(transformer)

    mnist_train = torchvision.datasets.CIFAR10(root=root, train=True,download=True,transform=transformer)
    mnist_test = torchvision.datasets.CIFAR10(root=root, train=False,download=True,transform=transformer)
    num_workers = 0 if sys.platform.startswith('win32') else 4

    train_iter = torch.utils.data.DataLoader(mnist_train,
                                              batch_size, shuffle=True,
                                              num_workers=num_workers)
    test_iter = torch.utils.data.DataLoader(mnist_test,
                                            batch_size, shuffle=False,
                                            num_workers=num_workers)
    return train_iter, test_iter

def load_data_fashion_mnist(batch_size, resize=None, root=os.path.join(
    '~', '.pytorch', 'datasets', 'fashion-mnist')):
    """Download the Fashion-MNIST dataset and then load into memory."""
    root = os.path.expanduser(root)
    transformer = []
    if resize:
        transformer += [torchvision.transforms.Resize(resize)]
    transformer += [torchvision.transforms.ToTensor()]
    transformer = torchvision.transforms.Compose(transformer)

    mnist_train = torchvision.datasets.FashionMNIST(root=root, train=True,download=True,transform=transformer)
    mnist_test = torchvision.datasets.FashionMNIST(root=root, train=False,download=True,transform=transformer)
```

```

num_workers = 0 if sys.platform.startswith('win32') else 4

train_iter = torch.utils.data.DataLoader(mnist_train,
                                          batch_size, shuffle=True,
                                          num_workers=num_workers)
test_iter = torch.utils.data.DataLoader(mnist_test,
                                         batch_size, shuffle=False,
                                         num_workers=num_workers)

return train_iter, test_iter

# funções básicas
def _get_batch(batch):
    """Return features and labels on ctx."""
    features, labels = batch
    if labels.type() != features.type():
        labels = labels.type(features.type())
    return (torch.nn.DataParallel(features, device_ids=devices_ids),
            torch.nn.DataParallel(labels, device_ids=devices_ids), features.shape[0])

# Função usada para calcular acurácia
def evaluate_accuracy(data_iter, net, loss):
    """Evaluate accuracy of a model on the given data set."""

    acc_sum, n, l = torch.Tensor([0]), 0, 0
    net.eval()
    with torch.no_grad():
        for X, y in data_iter:
            #y = y.astype('float32')
            X, y = X.to(device), y.to(device)
            y_hat = net(X)
            l += loss(y_hat, y).sum()
            acc_sum += (y_hat.argmax(axis=1) == y).sum().item()
            n += y.size()[0]

    return acc_sum.item() / n, l.item() / len(data_iter)

# Função usada no treinamento e validação da rede
def train_validate(net, train_iter, test_iter, batch_size, trainer, loss,
                  num_epochs):
    print('training on', device)
    for epoch in range(num_epochs):
        net.train()
        train_l_sum, train_acc_sum, n, start = 0.0, 0.0, 0, time.time()
        for X, y in train_iter:
            X, y = X.to(device), y.to(device)
            y_hat = net(X)
            trainer.zero_grad()
            l = loss(y_hat, y).sum()
            l.backward()
            trainer.step()
            train_l_sum += l.item()
            train_acc_sum += (y_hat.argmax(axis=1) == y).sum().item()
            n += y.size()[0]
        test_acc, test_loss = evaluate_accuracy(test_iter, net, loss)
        print('epoch %d, train loss %.4f, train acc %.3f, test loss %.4f, '
              'test acc %.3f, time %.1f sec'
              % (epoch + 1, train_l_sum / len(train_iter), train_acc_sum / n, test_loss,
                 test_acc, time.time() - start))

```

Detalhes Técnicos

Em 2015, uma heurística inteligente chamada [batch normalization](#) provou ser imensamente útil para melhorar a confiabilidade e a velocidade de convergência no treinamento de modelos profundos. Em cada iteração de treinamento, o [batch normalization](#) normaliza as ativações de cada neurônio da camada oculta (em cada camada onde é aplicada) subtraindo sua média e dividindo pelo seu desvio padrão, estimando ambos com base no mini-batch atual. Note que se o tamanho do batch fosse \$1\$, não aprenderíamos nada porque durante o treinamento, todos os \$n\$ levaria valor \$0\$. No entanto, com minibatches grandes o suficiente, a abordagem se prova eficaz e estável.

Em poucas palavras, a ideia do [batch normalization](#) é transformar a ativação em uma determinada camada de \$\mathbf{x}\$ para:

$$\mathbf{BN}(\mathbf{x}) = \gamma \odot \frac{\mathbf{x} - \hat{\mu}}{\hat{\sigma}} + \beta$$

Aqui, \$\hat{\mu}\$ é a estimativa da média e \$\hat{\sigma}\$ é a estimativa da variância. O resultado é que as ativações são aproximadamente reescaladas para uma média zero e uma variância unitária. Como podemos notar, as ativações das camadas intermediárias não pode divergir muito pois estamos ativamente redimensionando-as de volta para uma dada ordem de grandeza através de \$\mu\$ e \$\sigma\$. Entretanto, em alguns casos, as ativações pode precisar diferir dos dados

padronizados. Para lidar com isso e dar mais liberdade à essa normalização, definimos um coeficiente de escala de coordenadas γ e um offset β . Intuitivamente, espera-se que essa normalização nos permita ser mais agressivo ao escolher taxas de aprendizado maiores.

Em princípio, podemos querer usar todos os nossos dados de treinamento para estimar a média e variância. No entanto, as ativações correspondentes a cada exemplo mudam cada vez que atualizamos nosso modelo. Para remediar este problema, o [batch normalization](#) usa apenas o minibatch atual para estimar $\hat{\mu}$ e $\hat{\sigma}$. É justamente por esse fato de normalizarmos com base apenas no *batch* atual que o método se chama *batch normalization*. Para indicar qual *minibatch* $\mathcal{B}$ é usado, nós denotamos essas variáveis como $\hat{\mu}_{\mathcal{B}}$ e $\hat{\sigma}_{\mathcal{B}}$.

$$\hat{\mu}_{\mathcal{B}} \leftarrow \frac{1}{|\mathcal{B}|} \sum_{\mathbf{x} \in \mathcal{B}} \mathbf{x}$$

$$\hat{\sigma}_{\mathcal{B}}^2 \leftarrow \frac{1}{|\mathcal{B}|} \sum_{\mathbf{x} \in \mathcal{B}} (\mathbf{x} - \hat{\mu}_{\mathcal{B}})^2 + \epsilon$$

Observe que adicionamos uma pequena constante $\epsilon > 0$ à estimativa de variância para garantir que nunca acabemos por dividir por zero, mesmo nos casos em que a estimativa de variância pode desaparecer por acidente. Vamos agora ver, na prática, o uso do [batch normalization](#).

Camadas de Batch Normalization

O [batch normalization](#) para camadas totalmente conectadas (*fully-connected* ou Densas) e camadas convolucionais são ligeiramente diferentes. Isso se deve à dimensionalidade dos dados gerados pelas camadas convolucionais. Os dois casos são discutidos abaixo. Observe que uma das principais diferenças entre a camada de [batch normalization](#) e outras camadas é que a primeira opera em um *minibatch* completo de cada vez (caso contrário, não é possível calcular os parâmetros de média e variância por *batch*).

Camadas densas

Normalmente, aplicamos a camada de [batch normalization](#) entre a transformação e a função de ativação em uma camada densa. A seguir, denotamos por $\mathbf{u}$ a entrada e por $\mathbf{x} = \mathbf{W}\mathbf{u} + \mathbf{b}$ a saída da transformada linear. Isso produz a seguinte fórmula:

$$\mathbf{y} = \phi(\text{BN}(\mathbf{x})) = \phi(\text{BN}(\mathbf{W}\mathbf{u} + \mathbf{b}))$$

Lembre-se de que a média e a variância são calculadas no **mesmo minibatch** $\mathcal{B}$ no qual a transformação é aplicada. Lembre-se também que o coeficiente γ e o offset β são parâmetros que precisam ser aprendidos. Eles garantem que o efeito do [batch normalization](#) possa ser neutralizado conforme necessário.

Camadas convolucionais

Para camadas convolucionais, o [batch normalization](#) ocorre após o cálculo da convolução e antes da aplicação da função de ativação. Se a computação de convolução gerar múltiplos canais, realizamos o [batch normalization](#) para **cada** uma das saídas desses canais, que tem parâmetros (γ e β) independentes. Suponha que haja exemplos de m no *batch*. Em um único canal, assumimos que a altura e a largura da saída da convolução são p e q , respectivamente. Precisamos realizar o [batch normalization](#) para $m \times p \times q$ elementos neste canal simultaneamente. Ao executar o cálculo de padronização para esses elementos, usamos a mesma média e variância. Em outras palavras, usamos as médias e as variâncias dos elementos $m \times p \times q$ neste canal em vez de um por pixel.

Pytorch e o caso de estudo LeNet-5

Agora vamos implementar a [LeNet-5](#) usando a camada de [batch normalization](#).

Em frameworks modernos, camadas de [batch normalization](#) já vem implementadas e são fáceis de usar.

```
# parâmetros: número de epochs, learning rate (ou taxa de aprendizado),
# tamanho do batch, e lambda do weight decay
num_epochs, lr, batch_size, wd_lambda = 10, 0.01, 128, 0.0001

# rede baseada na LeNet-5
net = nn.Sequential(
    nn.Conv2d(in_channels=1, out_channels=6, kernel_size=5, stride=1, padding=0), # entrada: (b, 1, 32, 32) e saída: (
    nn.BatchNorm2d(num_features=6),
    nn.Tanh(),
    nn.AvgPool2d(kernel_size=2, stride=2, padding=0), # entrada: (b, 6, 28, 28) e saída: (b, 6, 14, 14)
    nn.Conv2d(in_channels=6, out_channels=16, kernel_size=5, stride=1, padding=0), # entrada: (b, 6, 14, 14) e saída:
    nn.BatchNorm2d(num_features=16),
    nn.Tanh(),
    nn.AvgPool2d(kernel_size=2, stride=2, padding=0), # entrada: (b, 16, 10, 10) e saída: (b, 16, 5, 5)
    nn.Conv2d(in_channels=16, out_channels=120, kernel_size=5, stride=1, padding=0), # entrada: (b, 16, 5, 5) e saída:
    nn.BatchNorm2d(num_features=120),
    nn.Tanh(),
    nn.Flatten(), # lineariza formando um vetor # entrada: (b, 120, 1, 1) e saída: (b, 120*1*1) = (b, 120)
    nn.Linear(120, 84),
```

```

nn.BatchNorm1d(84),
nn.Tanh(),
nn.Linear(84, 10))

# Sending model to device
net.to(device)
print(summary(net,(1,32,32))) # visualize number of parameters' net, output of each layer and total mega bytes necessary for
# and stored weights.

# função de custo (ou loss)
loss = nn.CrossEntropyLoss()

# carregamento do dado: mnist
train_iter, test_iter = load_data_fashion_mnist(batch_size, resize=32)

# trainer do gluon
trainer = optim.Adam(net.parameters(), lr=lr, weight_decay=wd_lambda)

# treinamento e validação via Pytorch
train_validate(net, train_iter, test_iter, batch_size, trainer, loss,
               num_epochs)

```

```

-----
Layer (type)          Output Shape          Param #
-----
Conv2d-1              [-1, 6, 28, 28]       156
BatchNorm2d-2         [-1, 6, 28, 28]       12
Tanh-3                [-1, 6, 28, 28]       0
AvgPool2d-4           [-1, 6, 14, 14]       0
Conv2d-5              [-1, 16, 10, 10]      2,416
BatchNorm2d-6         [-1, 16, 10, 10]      32
Tanh-7                [-1, 16, 10, 10]      0
AvgPool2d-8           [-1, 16, 5, 5]        0
Conv2d-9              [-1, 120, 1, 1]       48,120
BatchNorm2d-10        [-1, 120, 1, 1]       240
Tanh-11               [-1, 120, 1, 1]       0
Flatten-12            [-1, 120]              0
Linear-13              [-1, 84]               10,164
BatchNorm1d-14        [-1, 84]               168
Tanh-15               [-1, 84]               0
Linear-16              [-1, 10]               850
=====
total params: 62,158
Trainable params: 62,158
Non-trainable params: 0
-----

```

Input size (MB): 0.00

Forward/backward pass size (MB): 0.16

Params size (MB): 0.24

Estimated Total Size (MB): 0.40

None

Downloading <http://fashion-mnist.s3-website.eu-central-1.amazonaws.com/train-images-idx3-ubyte.gz> to /root/.pytorch/datasets/fashion-mnist/train-images-idx3-ubyte.gz [26427392/? [00:20<00:00, 6685120.65it/s]

Extracting /root/.pytorch/datasets/fashion-mnist/FashionMNIST/raw/train-images-idx3-ubyte.gz to /root/.pytorch/datasets/fashion-mnist/FashionMNIST/raw/train-images-idx3-ubyte.gz [26427392/? [00:20<00:00, 6685120.65it/s]

Downloading <http://fashion-mnist.s3-website.eu-central-1.amazonaws.com/train-labels-idx1-ubyte.gz> to /root/.pytorch/datasets/fashion-mnist/train-labels-idx1-ubyte.gz [32768/? [00:00<00:00, 35676.45it/s]

Extracting /root/.pytorch/datasets/fashion-mnist/FashionMNIST/raw/train-labels-idx1-ubyte.gz to /root/.pytorch/datasets/fashion-mnist/FashionMNIST/raw/train-labels-idx1-ubyte.gz [32768/? [00:00<00:00, 35676.45it/s]

Downloading <http://fashion-mnist.s3-website.eu-central-1.amazonaws.com/t10k-images-idx3-ubyte.gz> to /root/.pytorch/datasets/fashion-mnist/t10k-images-idx3-ubyte.gz [4423680/? [00:16<00:00, 780219.72it/s]

Extracting /root/.pytorch/datasets/fashion-mnist/FashionMNIST/raw/t10k-images-idx3-ubyte.gz to /root/.pytorch/datasets/fashion-mnist/FashionMNIST/raw/t10k-images-idx3-ubyte.gz [4423680/? [00:16<00:00, 780219.72it/s]

Downloading <http://fashion-mnist.s3-website.eu-central-1.amazonaws.com/t10k-labels-idx1-ubyte.gz> to /root/.pytorch/datasets/fashion-mnist/t10k-labels-idx1-ubyte.gz [8192/? [00:00<00:00, 9671.50it/s]

Extracting /root/.pytorch/datasets/fashion-mnist/FashionMNIST/raw/t10k-labels-idx1-ubyte.gz to /root/.pytorch/datasets/fashion-mnist/FashionMNIST/raw/t10k-labels-idx1-ubyte.gz [8192/? [00:00<00:00, 9671.50it/s]

Processing...

/usr/local/lib/python3.6/dist-packages/torchvision/datasets/mnist.py:480: UserWarning: The given NumPy array is not writeable

return torch.from_numpy(parsed.astype(m[2], copy=False)).view(*s)

Done!

training on cpu

epoch 1, train loss 0.5183, train acc 0.808, test loss 0.5014, test acc 0.800, time 33.2 sec

epoch 2, train loss 0.3614, train acc 0.868, test loss 0.3754, test acc 0.858, time 32.2 sec

epoch 3, train loss 0.3266, train acc 0.881, test loss 0.3322, test acc 0.876, time 32.3 sec

epoch 4, train loss 0.3098, train acc 0.887, test loss 0.4106, test acc 0.861, time 32.9 sec

epoch 5, train loss 0.2985, train acc 0.891, test loss 0.6496, test acc 0.796, time 32.1 sec

epoch 6, train loss 0.2902, train acc 0.895, test loss 0.4986, test acc 0.792, time 32.3 sec

epoch 7, train loss 0.2809, train acc 0.897, test loss 0.3799, test acc 0.864, time 32.4 sec

epoch 8, train loss 0.2757, train acc 0.899, test loss 0.3364, test acc 0.868, time 32.7 sec

epoch 9, train loss 0.2695, train acc 0.902, test loss 0.2998, test acc 0.889, time 32.4 sec

epoch 10, train loss 0.2651, train acc 0.902, test loss 0.3798, test acc 0.865, time 32.4 sec